COMPUTER SCIENCE EDUCATION

Fractal

A Beginner-Friendly Programming Language

A bridge between Python and C, designed to smooth the transition from introductory to systems-level programming

Beginner Friendly
Compiled Language

Educational
Learning Kit

Adwaidh Payattuparambil,
Harikrishnan R
Theerttha S
Government Engineering College, Thrissur
March 2026

The Challenge in Computing Education

1 The Python-to-C Gap

Computing education starts with Python, then jumps to C and the gap between them is significant.

2 Python's Hidden Complexity

Python hides key concepts like type enforcement and memory management, leaving students under-prepared.

3 C's Steep Learning Curve

C's strict syntax and poor error messages make it feel foreign to students with no transitional foundation.

The Solution

Fractal addresses this gap by adopting C-like syntax while providing learner-oriented features absent in C.

- Descriptive, actionable error messages
- Visible type annotations
- Integrated learning environment

Critical Barriers in Programming Education

01

Lack of Type Discipline in Python

Conceptual gap in type understanding

02

Cryptic Compiler Diagnostics

Active impediment to learning

03

Absence of Pre-Configured Teaching Environment

Discourages experimentation

04

No Execution Visibility

Weak mental model formation

05 High Installation Barrier

Project Goals & Deliverables

Language Design

Compiler Pipeline

Testing & Validation

Demonstrate correctness by compiling and executing programs implementing a range of algorithms and data structures.

- ✓ Fibonacci sequence
- ✓ Armstrong numbers
- ✓ Palindrome detection
- ✓ Matrix operations
- ✓ Binary search trees
- ✓ Dijkstra's algorithm

Comprehensive Error Messages

Full-Featured Development Env

One Command Install

Automated CI/CD Pipeline

Landing Page

Existing Programming languages

1 Rust

+ Good Error Messages, Fast Execution.
- Overly Opinionated, Difficult to master.

2 Go

+ Simple Clean Syntax, No Pointers.
- No Dedicated IDE, Complicated Setup.

Fractal combines the best of both worlds to give learners the tools to lay their foundation.

Fractal Language Design

Dual-Prefix Visual Grammar

```
!func fibonacci(:int n) -> :int {
    !if (n <= 0){
        !return 1;
    }
    !return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Transpilation to Rust

Type System

Fractal's type system maps one-to-one to C types, providing clear conceptual alignment:

`:int`	→ long (64-bit signed)
`:float`	→ double (64-bit)
`:char`	→ char (single character)
`:boolean`	→ bool (true/false)
`:array`	→ Fixed-size array
`:list`	→ Dynamic grow-able list
`:struct`	→ User-defined record

Key Innovation

The dual-prefix grammar makes token categories immediately visible, eliminating the need for contextual knowledge to

Compiler Pipeline and IDLE Features

Six-Phase Compiler Pipeline

1. **Preprocessor**
   Resolves !import directives, handles module splicing
2. **Lexer**
   Converts source text to typed tokens
3. **Parser**
   Recursive-descent LL(1) parser constructs AST
4. **Semantic Analyzer**
   Type checking, scope resolution, struct validation
5. **Return Checker**
   Verifies all execution paths return values
6. **Code Generator**
   Emits Rust source code for rustc compilation

Automated CI/CD Pipeline

GitHub Actions workflow builds and releases versioned binaries on every commit to main, ensuring the website always offers the latest version.

IDLE Editor Features

- Syntax Highlighting
- Step-Through Debugger
- Variable Inspector
- AST Viewer
- Built-in Documentation
- Find and Replace
- Theme Control
- Integrated Terminal

One-Command Installation

```
wget -O install.sh https://raw.githubusercontent.com/Pixelrick420/Fractal/main/executable/install.sh &&
sudo bash install.sh
```

Automates Rust installation, fractal compiler and editor binary download, ELF verification, and PATH configuration.

Testing Outcomes and Performance

Compilation Pipeline Results

The compiler successfully processes all 14 programs in the test suite, covering data structures, algorithms, and mathematical utilities.

14	<2s	100%
Programs Tested	Compilation Time	Success Rate

Test Programs

- ✓ Fibonacci series
- ✓ Armstrong numbers
- ✓ Palindrome check
- ✓ Matrix multiplication
- ✓ Binary search tree
- ✓ Dijkstra's algorithm

Error Message Quality

Quality difference compared to GCC is significant.

Debugger and Visualizer Tools

Enhance the visual learning experience for the user.

Project Website

https://fractal-programming.vercel.app/ serves as distribution hub with live release version from GitHub API.

Planned Enhancements

- Cross-Platform Support
- Online Playground
- Standard Library
- Formal Learning Study
- Package Manager

Key Achievements & Impact

- Complete Compiler Pipeline
- Innovative Language Design
- Comprehensive Learning Environment

Impact

Fractal demonstrates that the gap between Python and C can be addressed at the language-design and tooling level without sacrificing correctness or performance.

Students exposed to:
Static typing, control flow, structs

Visual learning through:
AST viewer, variable inspector

C-compatible syntax:
Before encountering pointers

Open Source

Released as free, open-source software suitable for adoption in introductory programming courses worldwide.

Thank You